

Production Order List — Screen Processing Design (画面処理設計)

002_DD-SPD — elaborates 002_DD, implements 002_BD, requirements REQ-020–REQ-027.

Document control (改版履歴)

Field	Value
Document ID	002_DD-SPD
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-003
Created by	Claude (for ThanhTN)
Created date	2026-09-22
Last updated by	Claude (for ThanhTN)
Last updated date	2026-09-22

Version	Date	Author	Revision content
1	2026-09-22	Claude (for ThanhTN)	Initial creation
2	2026-09-22	Claude (for ThanhTN)	WI-004 (DEC-016): the page renders inside the shared <code>AppHeader</code> , which now contains the navbar (<code>AppNavbar</code> , designed in 003_DD module 10 and 003_DD-SPD §8; breadcrumb links become <code>GuardedLink</code> , 003_DD module 11). No processing step of this screen changes
3	2026-09-23	Claude (for ThanhTN)	WI-005: the UI is Japanese. Page title, button names, paging text and formatting helpers updated to the Japanese catalog (WI-005 DEC-003, DEC-006, DEC-007). No field, rule, API or processing changes

Overview and process list

Field	Value
Screen / file name	<code>src/frontend/src/features/production-orders/ProductionOrderListPage.tsx</code> and the components it composes; the server side of the request is in <code>ProductionOrdersController.List</code>
Overview	How SCR-002 reads its view state from the URL, queries, renders each state, and reacts to filtering, sorting, paging and navigation — one block per component

No	Process name	Overview	Notes
1	<code>ProductionOrderListPage</code> — P-10 Load and query	Role gate, view state from the URL, product options, query, state selection	E-10, E-19
2	<code>useListViewState</code> — P-11 View-state synchronization	URL ⇌ state, defaults, history entries	E-10–E-15, E-19
3	<code>ProductionOrderFilters</code> — P-12 Filter, search and clear	Local filter values, client validation, submit	E-11, E-12
4	<code>ProductionOrderTable</code> — P-13 Sort and row activation	Header buttons, <code>aria-sort</code> , row link and mouse row click	E-13, E-16
5	<code>ListPagination</code> — P-14 Page and page size	Prev/Next bounds, page-size change	E-14, E-15
6	<code>ProductionOrderListPage</code> — P-15 Query-response handling	Mapping of each HTTP outcome to a screen state	E-10, E-18
7	<code>ProductionOrdersController.List</code> — request boundary	Auth, query binding and validation, result → HTTP	Server steps are in 002_DD-FN

Reference documents

No	Document	Purpose / use	Notes
1	002_DD	Items, states, message catalog, module list	Parent document
2	002_DD-API	The endpoint contract called below	
3	002_DD-FN	Server-side steps (<code>ListAsync</code> , repository methods)	
4	002_BD	Events E-10–E-19, validation V-09–V-13, states	
5	001_DD-SPD	SCR-001's processing, reached from this screen	Navigation target

Processing design

1. ProductionOrderListPage — P-10 Load and query

Field	Value
Detail	Route component for <code>/production-orders</code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: the URL is the single source of truth for what is displayed. The page reads the view state from it, fires exactly one list query per distinct view state, and renders one of five mutually exclusive states. Every user action changes the URL; the query is a reaction to that change, never a second, parallel path — so Back, Forward, reload and a pasted link all go through the same code as a button press.

Used components / services

No	Name	Overview	Notes
1	<code>useAuth</code>	Current user and roles	WI-001
2	<code>useListViewState</code>	View state from the URL	\$2
3	<code>api.ts</code> (<code>listOrders</code> , <code>listProducts</code>)	Typed wrappers over <code>apiClient</code>	002_DD module 10
4	<code>ProductionOrderFilters</code> , <code>ProductionOrderTable</code> , <code>ListPagination</code> , <code>MessageBanner</code>	Rendered children	<code>MessageBanner</code> reused from 001_DD

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	Read the view state from the URL	Unreadable or out-of-range value → that parameter falls back to its default and the URL is rewritten once with <code>replace</code>	\$2	No error page (REQ-027)
2	Role gate on <code>user.roles</code>	Neither <code>Admin</code> nor <code>Operator</code> → stop	—	<code>forbidden</code> panel (MSG-E020), no API call
3	Fetch the product options once per mount	Failure → product filter disabled with a load error; the rest of the screen still works	<code>GET /api/products</code>	Other filters remain usable

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
4	Set state <code>loading</code> ; run the list query for the current view state	A view state identical to the one in flight is not re-queried; a new one supersedes the old, whose response is ignored	<code>GET /api/production-orders</code>	Loading indicator replaces the table body, <code>aria-busy="true"</code> ; previous rows are not presented as the result
5	Handle the response	See §6	§6	<code>ready</code> / <code>empty</code> / <code>noMatch</code> / <code>error</code>
6	Set <code>document.title</code> to 製造指示一覧 – ProductionManagementAI (Production orders)	—	—	—

Five render states, mutually exclusive: `loading` , `ready` (table with rows), `empty` (no orders exist at all, MSG-I003), `noMatch` (filters set, nothing matched, MSG-I004), `error` (banner MSG-E013 with **Retry**). `forbidden` is a sixth, reached only at step 2 or from a 403.

2. useListViewState — P-11 View-state synchronization

Field	Value
Detail	Hook owning the parse/serialize of the nine URL parameters (002_BD 0-3)
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: one place converts between the URL's query string and the typed view state, so the page never reads `location.search` itself and the API query is built from the same values that are displayed. Parsing is total — it never throws and never produces an invalid state; anything unrecognized falls back to that parameter's default.

Used components / services

No	Name	Overview	Notes
1	<code>useSearchParams</code>	react-router URL access	

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	Parse <code>status</code> (repeatable)	Unknown value → dropped; duplicates collapsed	—	<code>Set<Status></code>
2	Parse <code>productId</code>	Not a UUID → dropped	—	<code>string \ null</code>

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
3	Parse <code>dueFrom</code> , <code>dueTo</code>	Malformed → dropped; <code>from > to</code> → both kept and shown with the V-10 error, since silently dropping a value the user can see would be worse	—	<code>string \ null</code> each
4	Parse <code>orderNumber</code>	Trimmed; longer than 20 → truncated to 20	—	<code>string \ null</code>
5	Parse <code>sort</code> , <code>dir</code>	Outside the allow-list → defaults <code>dueDate</code> , <code>asc</code>	—	Enum values
6	Parse <code>page</code> , <code>pageSize</code>	<code>page < 1</code> or non-numeric → 1; <code>pageSize</code> outside {10,20,50,100} → 20	—	Integers
7	<code>setView(next, { push })</code> serializes back, omitting every parameter that equals its default	Filter/sort/page change → <code>push</code> (a history entry, so Back works). A fallback rewrite at step 1–6 → <code>replace</code>	—	URL updated; P-10 step 4 reacts

Omitting defaults keeps a shared link short and readable (`/production-orders?status=Draft` rather than nine parameters), and it means the plain route and the fully defaulted URL are the same view.

3. ProductionOrderFilters — P-12 Filter, search and clear

Field	Value
Detail	Filter panel (items 7–13), a real <code><form></code> submitted by 検索 (Search)
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: the panel holds its own draft values while the user edits, and only publishes them to the view state on submit (DEC-008). That is what makes "the URL reflects what is displayed" true — an unsubmitted draft is not displayed, so it is not in the URL.

Used components / services

No	Name	Overview	Notes
1	<code>validateFilters</code>	Client-side V-09, V-10	002_DD module 5
2	<code>messages.ts</code>	Message IDs → text	Extended catalog

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	Initialize the draft from the current view state	On every view-state change (e.g. Back) the draft re-bases, so the panel always shows what produced the rows	—	Fields filled
2	Status checkbox / product select / date / text edits update the draft only	—	—	No query
3	検索 (Search) (click or Enter in the panel) → run <code>validateFilters</code>	Invalid → show inline errors, move focus to the first invalid field, send no query and leave the rows as they are	\$002_DD module 5	Field errors (MSG-E015/E016/E017)
4	Valid → publish the draft to the view state with <code>page = 1</code> , keeping sort and page size	—	\$2 <code>setView(push)</code>	P-10 queries
5	クリア (Clear) (item 12) or 絞り込みをクリア (Clear filters) (item 25) → reset the draft to empty, publish with <code>page = 1</code>	Disabled when no filter is set	\$2	P-10 queries the unfiltered list

4. ProductionOrderTable — P-13 Sort and row activation

Field	Value
Detail	Results table (items 16–22, 27), or the card list at the SP breakpoint
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: a real `<table>` whose header cells contain buttons; `aria-sort` on the active column is the accessible equivalent of the ▲/▼ marker. Row activation is a genuine link, with the row-level mouse click as a convenience on top (DEC-009).

Used components / services

No	Name	Overview	Notes
1	<code>Link</code> (react-router)	Order-number cell link to SCR-001	
2	<code>statusLabels</code> , <code>labels</code> ; <code>formatDate</code> , <code>formatTimestamp</code> , <code>formatNumber</code>	Status label (M-05), other texts, and date and number formatting (M-10)	<code>messages.ts</code> ; <code>src/lib/format.ts</code> (WI-005)

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	Render one row per item; the order-number cell is a <code><Link to={ /production-orders/\${id} }></code>	<code>isOverdue</code> → render the overdue marker next to the due date (item 27)	—	Table body
2	Header button click/Enter/Space	Already the active key → flip <code>dir</code> ; otherwise set the key with <code>asc</code>	<code>\$2</code> <code>setView(push)</code> with <code>page = 1</code>	P-10 queries; <code>aria-sort</code> moves
3	Row mouse click	Ignored when the click ended a text selection, or landed on the link itself or another interactive element	<code>navigate(id)</code>	Same destination as the link
4	SP breakpoint	Table replaced by one card per order, the whole card being the link; the sort select replaces the header buttons	<code>\$2</code>	Same query behavior

Step 3 exists only for the mouse. Keyboard users reach step 1's link through normal tab order, so no behavior depends on the row handler — if it were removed, the screen would still be fully operable.

5. ListPagination — P-14 Page and page size

Field	Value
Detail	Result summary, page-size select and Previous/Next (items 14, 15, 23)
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: page bounds are computed from the response's `total` and `pageSize` , not from what the client asked for, so the controls stay correct even when a request was made against a stale view.

Used components / services: none beyond `$2`.

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	<code>lastPage = max(1, ceil(total / pageSize))</code> ; render 「{page} / {lastPage} ページ」 (Page {page} of {lastPage}) and the M-09 summary	<code>total = 0</code> → the summary is replaced by the empty or no-match state	—	—

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
2	< 前へ / 次へ > (Previous / Next), named 「前のページ」 / 「次のページ」 for assistive technology	Disabled (not hidden) at page 1 and at <code>lastPage</code> ; a page past <code>lastPage</code> still renders the controls so the user can go back	\$2 <code>setView(push)</code>	P-10 queries
3	Page-size change	Always resets <code>page</code> to 1	\$2 <code>setView(push)</code>	P-10 queries

6. ProductionOrderListPage — P-15 Query-response handling

Field	Value
Detail	One place mapping every HTTP outcome of the list query to a screen state
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: the client renders from the response's echoed `sort` , `dir` , `page` and `pageSize` , not from its own request, so a late or retried response can never leave the table, the summary and the sort marker disagreeing.

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	200 with <code>total > 0</code>	—	—	<code>ready</code> : rows, summary, paging; the summary's live region announces the new total
2	200 with <code>total = 0</code>	No filter set → <code>empty</code> (MSG-I003 + + 新規製造指示 (New production order)); any filter set → <code>noMatch</code> (MSG-I004 + 絞り込みをクリア (Clear filters))	—	Filter panel keeps its values in both cases
3	200 with <code>items = []</code> but <code>total > 0</code>	Page past the last one	—	<code>ready</code> with an empty body and working paging controls — not an error (REQ-024)
4	400	Attach each <code>errors</code> entry to its filter field by parameter name; a parameter with no visible field (e.g. <code>sort</code>) → error banner	—	Rows unchanged; MSG-E015–E019, MSG-E002
5	401	—	<code>apiClient</code> redirects	<code>/login</code>
6	403	—	—	<code>forbidden</code> panel (MSG-E020)

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
7	Network failure or 5xx	—	—	<code>error</code> banner MSG-E013 with 再試行 (Retry) (E-18), which re-runs the same query unchanged

7. ProductionOrdersController.List — request boundary

Field	Value
Detail	Server entry point for <code>GET /api/production-orders</code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Processing overview: bind the query string, validate it into a typed query object, delegate, and map the result to HTTP. No business rule and no query composition lives here — those are in 002_DD-FN.

Used components / services

No	Name	Overview	Notes
1	<code>ProductionOrderListQuery.TryCreate</code>	Validation and normalization	002_DD module 5
2	<code>ProductionOrderService.ListAsync</code>	The query use case	002_DD-FN §1
3	<code>ProductionOrderProblems.ToActionResult</code>	<code>Result<T></code> → HTTP, RFC 9457	Existing, 001_DD

Processing flow

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
1	Authorize	No session → 401; wrong role → 403	policy <code>ProductionOrderEditor</code>	Empty body, as 001_DD-API
2	Bind the query parameters into the raw request type, whose fields are all strings	No type-level binding failure is possible, so the existing <code>InvalidModelStateResponseFactory</code> never fires here: <code>page=abc</code> reaches <code>TryCreate</code> and gets MSG-E019, its designed message, rather than a generic binding error	—	—
3	<code>TryCreate</code> validates and normalizes	Any failure → 400 with every offending parameter reported together	module 5	—

Step	Description	Branch / condition	Calls	Result (state / redirect / render)
4	Delegate	—	ListAsync	—
5	Map the result	Ok → 200; Invalid → 400	ToActionResult	—

Unresolved decisions

None.